Pigeonhole Principle Simulation

Complete Project Architecture, Data Structures & Implementation Report

Subject: Discrete Mathematics & Data Structures	**Language / Stack:** C++ (C++17), JS, HTML5, CSS3
Backend: Winsock2 Socket Server & DSA Engine	**Frontend:** Vanilla JS Interactive Visualizer

1. Executive Summary

This project implements a full-stack educational laboratory demonstrating the **Pigeonhole Principle**. The system separates concerns into a lightweight, high-performance C++ backend that handles discrete math calculations and data structures management, coupled with a responsive web frontend that visualizes the distribution of items into buckets in real-time.

2. Discrete Mathematical Foundations

Theorem (Pigeonhole Principle): If *n* items are put into *k* containers (where $n > k$), then at least one container must contain more than one item.

Generalized Pigeonhole Principle: If *n* items are distributed among *k* pigeonholes, then at least one pigeonhole must contain at least ⌈**n / k**⌉ items, calculated computationally as:

⌈ n / k ⌉ = ⌊ (n + k - 1) / k ⌋

Guaranteed minimum capacity in the maximally occupied bucket

3. Backend Data Structures (C++)

The C++ backend (backend.cpp) models the problem domain using standard Object-Oriented Data Structures patterned after Hash Tables with Separate Chaining:

Data Structure	DSA Concept	Description & Methods
struct Pigeon	Item Node	Encapsulates individual objects with a unique identifier (int id).
class Pigeonhole	Bucket / Container	Encapsulates a container holding dynamic item storage (std::vector<Pigeon>). Provides addPigeon(), getCount(), and hasCollision().
class PigeonholeTable	Hash Table / Bucket Array	The core mapping structure. Manages the array of buckets (vector<Pigeonhole>) and distribution lookup array (vector<int>).
std::vector<T>	Dynamic Array	Underlying container for buckets, occupants, and mapped assignments.
std::map<str, str>	Red-Black Tree Map	Used during HTTP query parsing for key-value extraction.

4. Algorithms & Complexity Analysis

Operation	Algorithm / Method	Time Complexity	Space Complexity
Bucket Hash	objectIndex % numHoles	**O(1)**	O(1)
Distribution	Sequential bucket insertion	**O(n)**	O(n + k)
Guarantee Calc	(n + k - 1) / k	**O(1)**	O(1)
Collision Check	Bucket occupancy count scan	**O(k)**	O(1)

5. System Architecture & REST Communication

Communication between the frontend client and C++ backend uses standard HTTP REST protocols over TCP sockets:

- **Client Endpoint Request:** GET /api/simulate?objects=25&boxes;=12
- **Winsock TCP Server:** Built with socket(), bind(), listen(), and accept() listening on port 8080.
- **JSON Response Structure:**

```
{ "objects": 8, "boxes": 5, "guarantee": 2, "assignments": [0, 1, 2, 3, 4, 0, 1, 2] }
```

6. Project Execution Guide

Step	Command / Action	Description
1. Compile C++	`g++ -std=c++17 backend.cpp -o backend.exe -lws2_32`	Compiles backend with Winsock library.
2. Run Backend	`.\backend.exe`	Launches HTTP API at port 8080.
3. Serve Frontend	`python -m http.server 3000`	Hosts web visualizer at port 3000.
4. Open Browser	`http://localhost:3000`	Access interactive UI in browser.